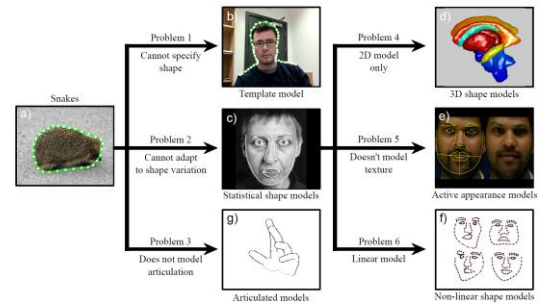
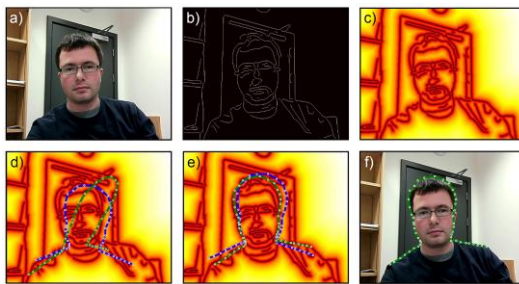


Relationships between models



Source: S. Prince

Shape template model



Local optimum

Global optimum

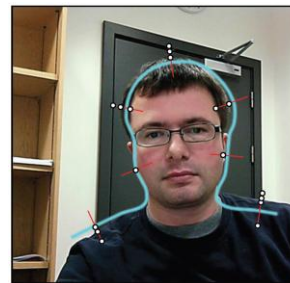
Blue initialization, green estimate

2/2/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Source: S. Prince

Iterative closest points



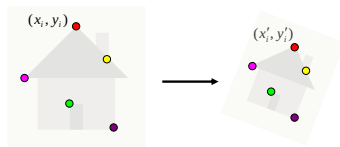
- Find nearest edge point to each landmark point
- Compute transformation in closed form
- Repeat

2/2/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Source: S. Prince

Fitting an affine transformation



$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

$$\begin{bmatrix} x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} x'_i \\ y'_i \\ \dots \end{bmatrix}$$

Source: R. Grammer

Least squares problem

A $m \times n$ matrix of rank n :

$$Ax = b$$

Solution with pseudo-inverse:

$$A^T(Ax - b) = 0$$

$$x = (A^T A)^{-1} A^T b$$

Pseudo-inverse A^+

Solution with SVD:

- SVD: $A = UDV^T$; $A^+ = VD^{-1}U^T$
- $b' = U^T b$
- $y_i = b'_i / D_{ii}$
- $x = Vy$

There are different ways to solve the problem. The optimal method depends on size and sparseness of A

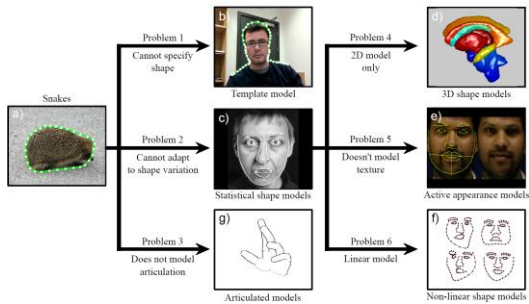
Specific C++ linear algebra libraries exist

2/2/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Relationships between models

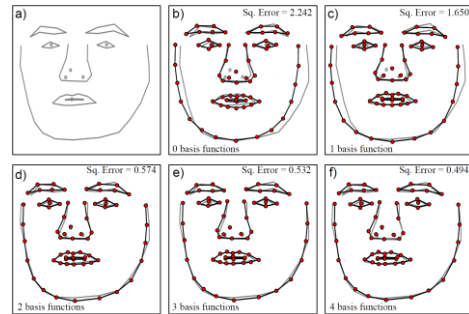
UNIVERSITÄT BONN



Source: S. Prince

Subspace shape model

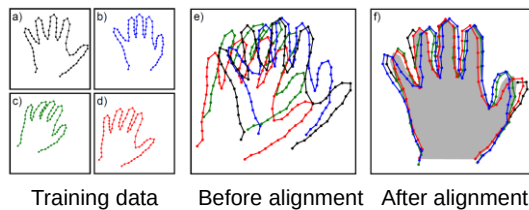
UNIVERSITÄT BONN



Source: S. Prince

Generalized Procrustes analysis

UNIVERSITÄT BONN



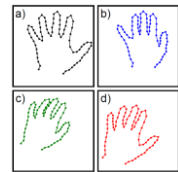
Training data Before alignment After alignment

Source: S. Prince

Generalized Procrustes analysis

UNIVERSITÄT BONN

- Given I shapes with N landmarks
- Correspondences between shapes are known (landmarks)
- Take one shape as mean shape
- Iterate:
 - Solve for affine transformation ψ_i for each shape i to the mean $A_i x = b_i$
 - Apply transformation ψ_i to each shape i
 - Compute the mean μ_n of each landmark (mean shape)



Source: S. Prince

Subspace shape model

UNIVERSITÄT BONN

- Generate data from model:

$$\mathbf{w}_i = \boldsymbol{\mu} + \Phi \mathbf{h}_i + \epsilon_i$$
 - $\boldsymbol{\mu}$ is the mean shape
 - the matrix $\Phi = [\phi_1, \phi_2, \dots, \phi_K]$ contains K basis functions in its columns
 - ϵ_i is normal noise with covariance $\sigma^2 \mathbf{I}$
- Can alternatively write

$$\mathbf{w}_i = \boldsymbol{\mu} + \sum_{k=1}^K \phi_k h_{ik} + \epsilon_i$$

Source: S. Prince

Approximating with subspace

UNIVERSITÄT BONN

Subspace model

$$\mathbf{w}_i = \boldsymbol{\mu} + \sum_{k=1}^K \phi_k h_{ik} + \epsilon_i$$

Can approximate a vector $\mathbf{w}$ with a weighted sum of the basis functions

$$\mathbf{w}_i \approx \boldsymbol{\mu} + \sum_{k=1}^K \phi_k h_{ik}$$

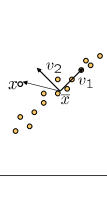
Surprising how well this works even with a small number of basis functions

Source: S. Prince

Linear subspaces

UNIVERSITÄT BONN

$\bar{x}$ is the mean of the orange points



Consider the variation along direction $\mathbf{v}$ among all of the orange points:

$$\text{var}(\mathbf{v}) = \sum_{\text{orange point } \mathbf{x}} \|(\mathbf{x} - \bar{\mathbf{x}})^T \cdot \mathbf{v}\|^2$$

What unit vector $\mathbf{v}$ minimizes var ?

$$\mathbf{v}_2 = \min_{\mathbf{v}} \{\text{var}(\mathbf{v})\}$$

What unit vector $\mathbf{v}$ maximizes var ?

$$\mathbf{v}_1 = \max_{\mathbf{v}} \{\text{var}(\mathbf{v})\}$$

$$\begin{aligned} \text{var}(\mathbf{v}) &= \sum_{\mathbf{x}} \|(\mathbf{x} - \bar{\mathbf{x}})^T \cdot \mathbf{v}\|^2 \\ &= \sum_{\mathbf{x}} \mathbf{v}^T (\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^T \mathbf{v} \\ &= \mathbf{v}^T \left[\sum_{\mathbf{x}} (\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^T \right] \mathbf{v} \\ &= \mathbf{v}^T \mathbf{A} \mathbf{v} \quad \text{where } \mathbf{A} = \sum_{\mathbf{x}} (\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^T \end{aligned}$$

Solution: $\mathbf{v}_1$ is eigenvector of $\mathbf{A}$ with largest eigenvalue
 $\mathbf{v}_2$ is eigenvector of $\mathbf{A}$ with smallest eigenvalue

2/10/2024

Georgios Gali - Institute of Computer Science III - Computer Vision Group

12

Principal component analysis (PCA)

UNIVERSITÄT BONN

- Given data points $\mathbf{w}_i$
- Compute mean μ
- $\mathbf{W} = (\mathbf{w}_1 - \mu, \dots, \mathbf{w}_n - \mu)$
- Eigenvalue decomposition:
 $\mathbf{W}\mathbf{W}^T = \mathbf{U}\Sigma\mathbf{V}^T\mathbf{U}^T = \mathbf{U}\mathbf{L}^2\mathbf{U}^T$ ($\mathbf{u}_i$ basis vectors)
- SVD: $\mathbf{W} = \mathbf{U}\Sigma\mathbf{V}^T$

10/02/2024

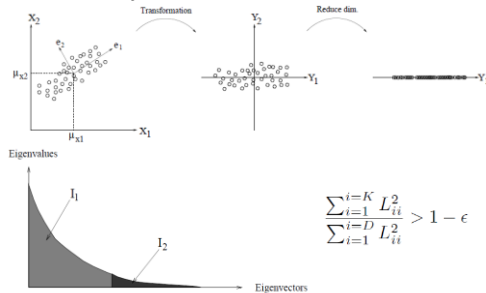
Georgios Gali - Institute of Computer Science III - Computer Vision Group

14

Principal component analysis (PCA)

UNIVERSITÄT BONN

- Dimensionality reduction



$$\frac{\sum_{i=1}^K L_{ii}^2}{\sum_{i=1}^D L_{ii}^2} > 1 - \epsilon$$

Source: T. Minka

2/10/2024

Georgios Gali - Institute of Computer Science III - Computer Vision Group

13

Learning PPCA

UNIVERSITÄT BONN

Learn parameters μ , Φ and σ^2 from data $\{\mathbf{w}_i\}_{i=1}^I$
 where $\mathbf{w}_i = [\mathbf{w}_{i1}^T, \mathbf{w}_{i2}^T, \dots, \mathbf{w}_{iN}^T]^T$.

$$\text{Learn mean: } \mu = \frac{\sum_{i=1}^I \mathbf{w}_i}{I}$$

Then set $\mathbf{W} = [\mathbf{w}_1 - \mu, \mathbf{w}_2 - \mu, \dots, \mathbf{w}_I - \mu]$
 and compute eigen-decomposition

$$\mathbf{W}\mathbf{W}^T = \mathbf{U}\mathbf{L}^2\mathbf{U}^T$$

Choose parameters

$$\hat{\sigma}^2 = \frac{1}{D-K} \sum_{j=K+1}^D L_{jj}^2$$

$$\hat{\Phi} = \mathbf{U}_K (\mathbf{L}_K^2 - \hat{\sigma}^2 \mathbf{I})^{1/2}$$

Source: S. Prince

2/10/2024

Georgios Gali - Institute of Computer Science III - Computer Vision Group

15

Simple version with PCA

UNIVERSITÄT BONN

1. Initialise the shape parameters, $\mathbf{b}$, to zero (the mean shape).
2. Generate the model point positions using $\mathbf{x} = \bar{\mathbf{x}} + \mathbf{P}\mathbf{b}$
3. Find the pose parameters (X_t, Y_t, s, θ) which best align the model points $\mathbf{x}$ to the current found points $\mathbf{Y}$

$$T_{X_t, Y_t, s, \theta} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} X_t \\ Y_t \end{pmatrix} + \begin{pmatrix} s \cos \theta & -s \sin \theta \\ s \sin \theta & s \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$
4. Project $\mathbf{Y}$ into the model co-ordinate frame by inverting the transformation T :

$$\mathbf{y} = T_{X_t, Y_t, s, \theta}^{-1}(\mathbf{Y})$$
5. Project $\mathbf{y}$ into the tangent plane to $\bar{\mathbf{x}}$ by scaling: $\mathbf{y}' = \mathbf{y}/(\mathbf{y} \cdot \bar{\mathbf{x}})$. (optional)
6. Update the model parameters to match to $\mathbf{y}'$

$$\mathbf{b} = \mathbf{P}^T(\mathbf{y}' - \bar{\mathbf{x}})$$
7. If not converged, return to step 2.

T. Cootes. An Introduction to Active Shape Models.

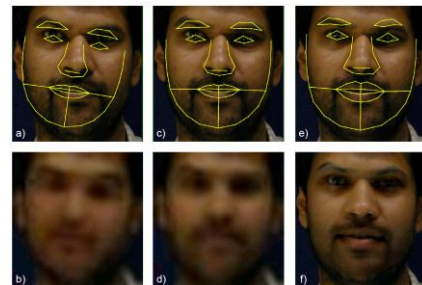
2/10/2024

Georgios Gali - Institute of Computer Science III - Computer Vision Group

17

Statistical models for shape and appearance

UNIVERSITÄT BONN



Source: T. Cootes

2/10/2024

Georgios Gali - Institute of Computer Science III - Computer Vision Group

18

Motivation: mosaics

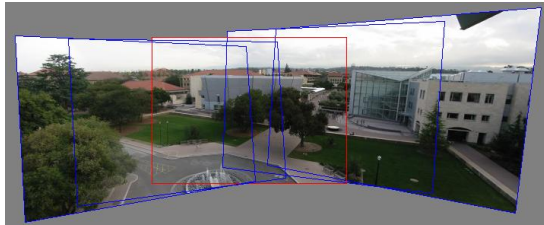


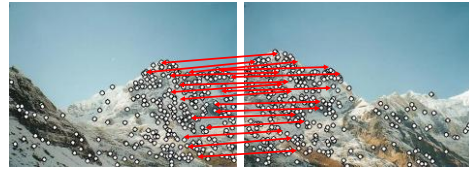
Image from <http://graphics.cs.cmu.edu/courses/15-463/2015/>, fall

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

18

Robust feature-based alignment



- Extract features
- Compute *putative matches*
- Loop:
 - Hypothesize transformation T (small group of putative matches that are related by T)
 - Verify transformation (search for other matches consistent with T)

Source: L. Leibe

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

19

Robust feature-based alignment



- Extract features
- Compute *putative matches*
- Loop:
 - Hypothesize transformation T (small group of putative matches that are related by T)
 - Verify transformation (search for other matches consistent with T)

Source: L. Leibe

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

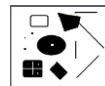
21

Corners as distinctive interest points



$$M = \sum w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$$

2 x 2 matrix of image derivatives (averaged in neighborhood of a point).



Notation:

$$I_x \Leftrightarrow \frac{\partial I}{\partial x}$$

$$I_y \Leftrightarrow \frac{\partial I}{\partial y}$$

$$I_x I_y \Leftrightarrow \frac{\partial I}{\partial x} \frac{\partial I}{\partial y}$$

Wolfram Orlandi

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

22

What does this matrix reveal?



Since M is symmetric, we have $M = X \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} X^T$



$$Mx_i = \lambda_i x_i$$

The *eigenvalues* of M reveal the amount of intensity change in the two principal orthogonal gradient directions in the window.

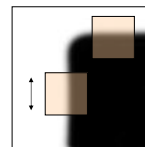
Wolfram Orlandi

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

23

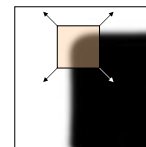
Corner response function



"edge":

$$\lambda_1 \gg \lambda_2$$

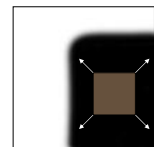
$$\lambda_2 \gg \lambda_1$$



"corner":

$$\lambda_1 \text{ and } \lambda_2 \text{ are large,}$$

$$\lambda_1 \sim \lambda_2;$$



"flat" region

$$\lambda_1 \text{ and } \lambda_2 \text{ are small;}$$

$$f = |M| - k (\text{Tr}(M))^2 = \lambda_1 \lambda_2 - k (\lambda_1 + \lambda_2)^2$$

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

24

Harris corner detector



- 1) Compute M matrix for each image window to get their *cornerness* scores.
- 2) Find points whose surrounding window gave large corner response ($f > \text{threshold}$)
- 3) Take the points of local maxima, i.e., perform non-maximum suppression

2/1/2026

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

25

Example of Harris application



2/1/2026

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

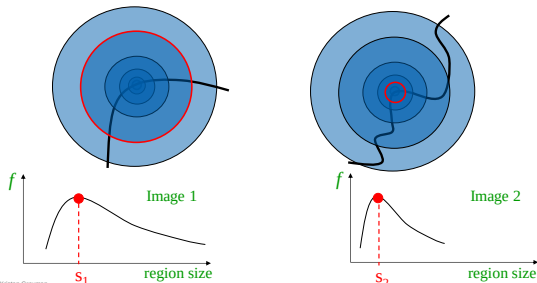
26

Automatic scale selection



Intuition:

- Find scale that gives local maxima of some function f in both position and scale.



Klaus Graumann

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

27

Scale invariant blob detection

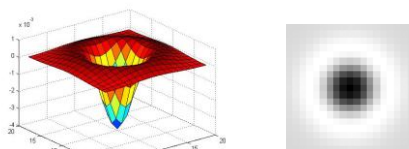


2/1/2026

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

28

Blob detection in 2D



- Laplacian of Gaussian: Circularly symmetric operator for blob detection in 2D

2/1/2026

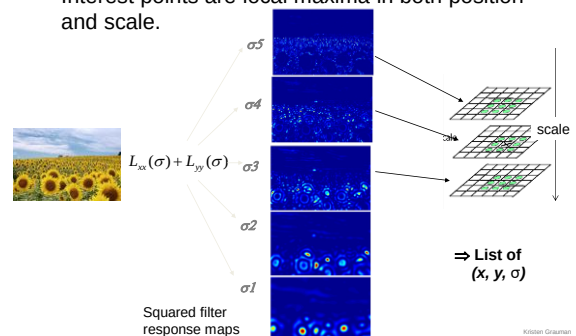
Jürgen Gall – Institute of Computer Science III – Computer Vision Group

29

Scale invariant interest points



Interest points are local maxima in both position and scale.



2/1/2026

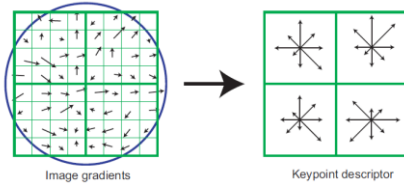
Jürgen Gall – Institute of Computer Science III – Computer Vision Group

30

SIFT descriptor [Lowe 2004]



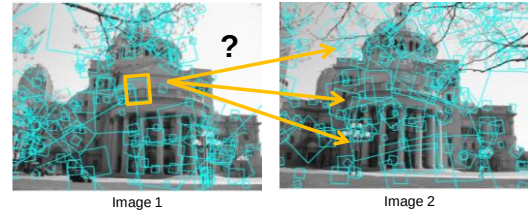
- 128 D vector



- Normalize to 1 (robust to illumination changes)
- Entries above 0.2 are set to 0.2 (reduce impact of very strong gradients e.g. shadow)
- Normalize to 1

2/10/2024 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 21

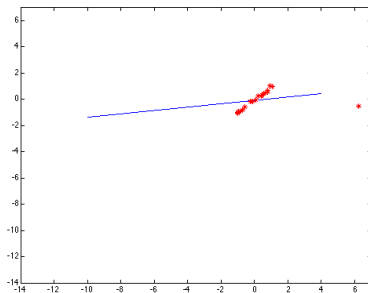
Matching local features



To add robustness to matching, can consider **ratio** :
 distance to best match / distance to second best match
 If low, first match looks good.
 If high and close to one, could be ambiguous match.

2/10/2024 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 22

Outliers affect least squares fit



2/10/2024 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 23

RANSAC: General form



RANSAC loop:

1. Randomly select a *seed group* of points on which to base transformation estimate (e.g., a group of matches)
2. Compute transformation from seed group
3. Find *inliers* to this transformation
4. If the number of inliers is sufficiently large, re-compute estimate of transformation on all of the inliers

Keep the transformation with the largest number of inliers

2/10/2024 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 24

How Many Samples?



With confidence p

$$P(\text{bad } k \text{ times}) = (1 - P(\text{good}))^k \leq 1 - p$$

$$k \log(1 - P(\text{good})) \leq \log(1 - p)$$

$$k \geq \log(1 - p) / \log(1 - P(\text{good}))$$

$$P(\text{good}) = \frac{\binom{I}{m}}{\binom{N}{m}} = \prod_{j=0}^{m-1} \frac{I - j}{N - j}$$

2/10/2024 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 25

Homography



$$(x, y) \rightarrow \begin{pmatrix} wx'/w & wy'/w \end{pmatrix} = (x', y')$$



$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$\mathbf{p}' \quad \mathbf{H} \quad \mathbf{p}$

2/10/2024 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 26

Solving for homographies

UNIVERSITÄT BONN

$$\begin{bmatrix} w x'_i \\ w y'_i \\ w \end{bmatrix} \cong \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

$$x'_i = \frac{h_{00}x_i + h_{01}y_i + h_{02}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$y'_i = \frac{h_{10}x_i + h_{11}y_i + h_{12}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$

$$y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x'_i & -y'_i & -x'_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -y'_i & -x'_i & -y'_i \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

2/10/2016

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

27

Solving for homographies

UNIVERSITÄT BONN

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1 & -y'_1 & -x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1 & -x'_1 & -y'_1 \\ & & & & & & & & \\ x_n & y_n & 1 & 0 & 0 & 0 & -x'_n & -y'_n & -x'_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -y'_n & -x'_n & -y'_n \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{matrix} A & h & 0 \\ 2n \times 9 & 9 & 2n \end{matrix}$$

Defines a least squares problem: minimize $\|Ah - 0\|^2$

- Since h is only defined up to scale, solve for unit vector $\hat{h}$
- Solution: $\hat{h}$ = eigenvector of $A^T A$ with smallest eigenvalue
- Works with 4 or more points

2/10/2016

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

28

Cameras

UNIVERSITÄT BONN



Lars Lindahl

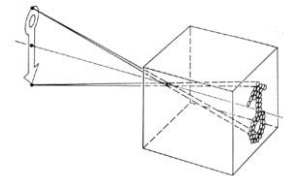
2/10/2016

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

29

Pinhole camera model

UNIVERSITÄT BONN



Pinhole model:

- Captures pencil of rays – all rays through a single point
- The point is called Center of Projection (focal point)
- The image is formed on the Image Plane

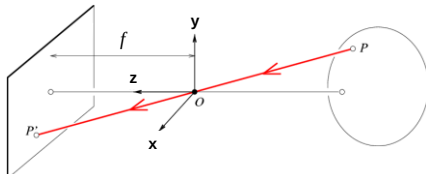
2/10/2016

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

30

Modeling projection

UNIVERSITÄT BONN



• Projection equations

- Compute intersection with image plane of ray from $P = (x, y, z)$ to O
- Derived using similar triangles

$$(x, y, z) \rightarrow \left(f \frac{x}{z}, f \frac{y}{z}, f\right)$$

- We get the projection by throwing out the last coordinate:

$$(x, y, z) \rightarrow \left(f \frac{x}{z}, f \frac{y}{z}\right)$$

2/10/2016

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

J. Peters, S. Sattler

31

Homogeneous coordinates

UNIVERSITÄT BONN

$$(x, y, z) \rightarrow \left(f \frac{x}{z}, f \frac{y}{z}\right)$$

Is this a linear transformation?

- no—division by z is nonlinear

Trick: add one more coordinate:

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

homogeneous image coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

homogeneous scene coordinates

Converting from homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w) \quad \begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow (x/w, y/w, z/w)$$

2/10/2016

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Stefan Sattler

32

Perspective Projection Matrix

Projection is a matrix multiplication using homogeneous coordinates

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z/f \\ 1 \end{bmatrix} \Rightarrow \left(f \frac{x}{z}, f \frac{y}{z}\right)$$

divide by the third coordinate

In practice: lots of coordinate transformations...

$$\begin{pmatrix} 2D \\ \text{point} \\ (3x1) \end{pmatrix} = \begin{pmatrix} \text{Camera to} \\ \text{pixel coord.} \\ \text{trans. matrix} \\ (3x3) \end{pmatrix} \begin{pmatrix} \text{Perspective} \\ \text{projection matrix} \\ (3x4) \end{pmatrix} \begin{pmatrix} \text{World to} \\ \text{camera coord.} \\ \text{trans. matrix} \\ (4x4) \end{pmatrix} \begin{pmatrix} 3D \\ \text{point} \\ (4x1) \end{pmatrix}$$

Lara Lindorff

2/10/2024

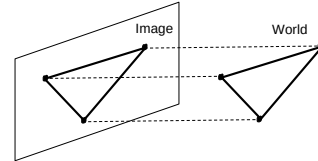
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

42

Orthographic Projection

Special case of perspective projection

- Distance from center of projection to image plane is infinite



- Also called "parallel projection"

- What's the projection matrix?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \\ 1 \end{bmatrix} \Rightarrow (x, y)$$

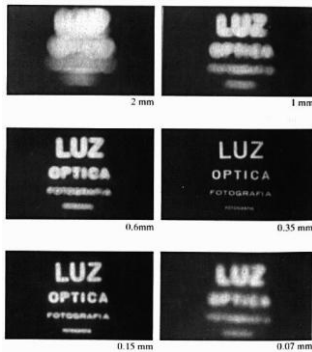
Stefan Seitz

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

43

Shrinking the aperture



Lara Lindorff

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

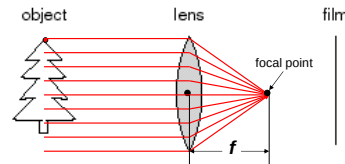
44

Adding a lens

A lens focuses light onto the film

- Thin lens model:

- Rays passing through the center are not deviated (pinhole projection model still holds)
- All parallel rays converge to one point on a plane located at the focal length f



Stefan Seitz

2/10/2024

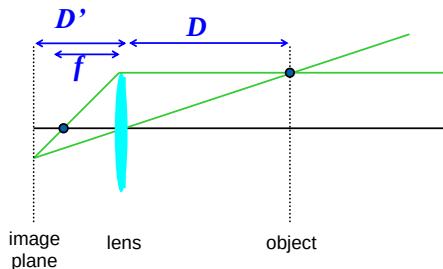
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

45

Thin lens formula

$$\frac{1}{D'} + \frac{1}{D} = \frac{1}{f}$$

Any point satisfying the thin lens equation is in focus.



Felix Dornig

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

46

Camera parameters

Intrinsic parameters

- Principal point coordinates
- Focal length
- Pixel magnification factors

$$K = \begin{bmatrix} m_x & & & f & p_x \\ & m_y & & f & p_y \\ & & & 1 & 1 \end{bmatrix} = \begin{bmatrix} \alpha_x & \beta_x \\ & \alpha_y & \beta_y \\ & & 1 & 1 \end{bmatrix}$$

Lindorff

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

47

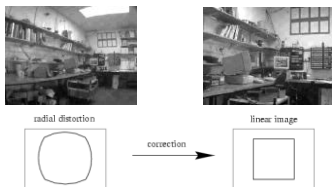
Camera parameters



Intrinsic parameters

- Principal point coordinates
- Focal length
- Pixel magnification factors
- Skew (non-rectangular pixels)
- Radial distortion

$$K = \begin{pmatrix} \alpha_x & \gamma & \beta_x \\ 0 & \alpha_y & \beta_y \\ 0 & 0 & 1 \end{pmatrix}$$



2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

18

Camera parameters



Intrinsic parameters

- Principal point coordinates
- Focal length
- Pixel magnification factors
- Skew (non-rectangular pixels)
- Radial distortion

Extrinsic parameters

- Rotation and translation relative to world coordinate system

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

19

Camera calibration



$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$

$$\begin{bmatrix} \lambda x \\ \lambda y \\ \lambda \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

2/10/2026

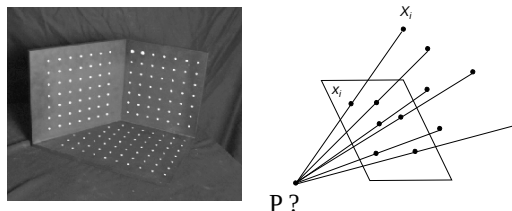
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Computer Vision

Camera calibration



Given n points with known 3D coordinates X_i and known image projections x_i , estimate the camera parameters



2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

20

Camera calibration: Linear method



$$\lambda x_i = P X_i \quad x_i \times P X_i = 0 \quad \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \times \begin{bmatrix} P_1^T X_i \\ P_2^T X_i \\ P_3^T X_i \end{bmatrix} = 0$$

$$\begin{bmatrix} 0 & -X_i^T & y_i X_i^T \\ X_i^T & 0 & -x_i X_i^T \\ -y_i X_i^T & x_i X_i^T & 0 \end{bmatrix} \begin{pmatrix} P_1 \\ P_2 \\ P_3 \end{pmatrix} = 0$$

Two linearly independent equations

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

21

Camera calibration: Linear method



$$\begin{bmatrix} 0^T & X_1^T & -y_1 X_1^T \\ X_1^T & 0^T & -x_1 X_1^T \\ \vdots & \vdots & \vdots \\ 0^T & X_n^T & -y_n X_n^T \\ X_n^T & 0^T & -x_n X_n^T \end{bmatrix} \begin{pmatrix} P_1 \\ P_2 \\ P_3 \end{pmatrix} = 0 \quad A p = 0$$

- P has 11 degrees of freedom (12 parameters, but scale is arbitrary)
- One 2D/3D correspondence gives us two linearly independent equations
- Homogeneous least squares
- 6 correspondences needed for a minimal solution

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

22

Camera calibration: Linear method



Advantages: easy to formulate and solve

Disadvantages

- Doesn't directly tell you camera parameters
- Doesn't model radial distortion
- Can't impose constraints, such as known focal length and orthogonality

Non-linear methods are preferred

- Define error as difference between projected points and measured points
- Minimize error using Newton's method or other non-linear optimization

2/10/2005 Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Intrinsic Calibration with Planes



Use only one plane

- Print a pattern on a paper
- Attach the paper on a planar surface
- Show the plane freely a few times to the camera

Advantages

- Flexible
- Robust

Implementation in OpenCV or Matlab toolbox:

http://www.vision.caltech.edu/bouguetj/calib_doc/

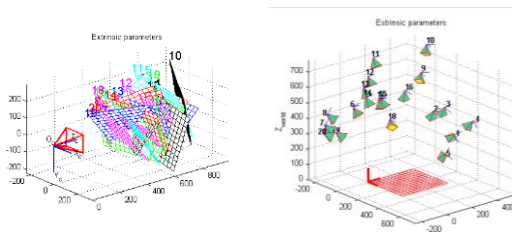
[Z. Zhang, Flexible Camera Calibration by Viewing a Plane from Unknown Orientations. ICCV99]

2/10/2005 Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Calibration process



Extrinsic parameters



2/10/2005 Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Camera Model



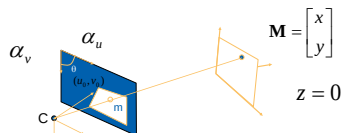
$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_A \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix}}_{[R \ t]} \underbrace{\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}}_M$$

2/10/2005 Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Plane projection



For convenience, assume the plane at $z = 0$.



The relation between image points and model points is then given by a homography $\mathbf{H}$:

$$\tilde{\mathbf{s}}\mathbf{m} = \tilde{\mathbf{H}}\mathbf{M} \quad \text{with} \quad \mathbf{H} = \mathbf{A} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \quad \mathbf{A} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix} \begin{bmatrix} x \\ y \\ 0 \\ 1 \end{bmatrix} = \mathbf{A} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2/10/2005 Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Linear Equations



Let

$$\mathbf{B} = \mathbf{A}^{-T} \mathbf{A}^{-1} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix} \quad \leftarrow \text{symmetric}$$

Define $\mathbf{b} = [B_{11} \ B_{22} \ B_{33} \ B_{12} \ B_{13} \ B_{23}]$ up to a scale factor

Rewrite

$$\mathbf{h}_1^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{h}_2 = 0$$

$$\mathbf{h}_1^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{h}_1 = \mathbf{h}_2^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{h}_2$$

as linear equations:

$$\begin{bmatrix} v_{12}^T \\ (v_{11} - v_{22})^T \end{bmatrix} \mathbf{b} = 0 \quad \mathbf{v}_{ij} = [h_{i1}h_{j1}, h_{i1}h_{j2} + h_{i2}h_{j1}, h_{i2}h_{j2}, h_{i3}h_{j1} + h_{i1}h_{j3}, h_{i3}h_{j2} + h_{i2}h_{j3}, h_{i3}h_{j3}]^T$$

2/10/2005 Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Camera parameters

UNIVERSITÄT BONN

Intrinsic camera parameters

$$\begin{aligned}
 v_0 &= (B_{12}B_{13} - B_{11}B_{23}) / (B_{11}B_{22} - B_{12}^2) \\
 \lambda &= B_{33} - [B_{13}^2 + v_0(B_{12}B_{13} - B_{11}B_{23})] / B_{11} \\
 \alpha &= \sqrt{\lambda / B_{11}} \\
 \beta &= \sqrt{\lambda B_{11} / (B_{11}B_{22} - B_{12}^2)} \\
 c &= -B_{12}\alpha^2\beta / \lambda \\
 u_0 &= cv_0 / \alpha - B_{13}\alpha^2 / \lambda
 \end{aligned}
 \quad
 \underbrace{\begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{A}}$$

Rotation and translation

$$\begin{aligned}
 \mathbf{r}_1 &= \lambda \mathbf{A}^{-1} \mathbf{h}_1, \quad \mathbf{r}_2 = \lambda \mathbf{A}^{-1} \mathbf{h}_2, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \quad \mathbf{t} = \lambda \mathbf{A}^{-1} \mathbf{h}_3 \\
 \lambda &= 1 / \|\mathbf{A}^{-1} \mathbf{h}_1\| = 1 / \|\mathbf{A}^{-1} \mathbf{h}_2\|
 \end{aligned}$$

2/10/2026

Dierksen, Gell - Institute of Computer Science III - Computer Vision Group

81

Distortion

UNIVERSITÄT BONN

Distortion model:

$$\begin{aligned}
 \tilde{x} &= x + x[k_1(x^2 + y^2) + k_2(x^2 + y^2)^2] \\
 \tilde{y} &= y + y[k_1(x^2 + y^2) + k_2(x^2 + y^2)^2]
 \end{aligned}$$

Centered at u_0 and v_0 :

$$\begin{aligned}
 \tilde{u} &= u + (u - u_0)[k_1(x^2 + y^2) + k_2(x^2 + y^2)^2] \\
 \tilde{v} &= v + (v - v_0)[k_1(x^2 + y^2) + k_2(x^2 + y^2)^2]
 \end{aligned}$$

Solution:

$$\underbrace{\begin{bmatrix} (u-u_0)(x^2+y^2) & (u-u_0)(x^2+y^2)^2 \\ (v-v_0)(x^2+y^2) & (v-v_0)(x^2+y^2)^2 \end{bmatrix}}_{\mathbf{D}} \underbrace{\begin{bmatrix} k_1 \\ k_2 \end{bmatrix}}_{\mathbf{d}} = \underbrace{\begin{bmatrix} \tilde{u}-u \\ \tilde{v}-v \end{bmatrix}}_{\mathbf{d}}$$

$$\mathbf{k} = (\mathbf{D}^T \mathbf{D})^{-1} \mathbf{D}^T \mathbf{d}$$

2/10/2026

Dierksen, Gell - Institute of Computer Science III - Computer Vision Group

82

Non-linear optimization

UNIVERSITÄT BONN

In practice, closed-form solution is used for initialization of non-linear optimization problem

$$\sum_{i=1}^n \sum_{j=1}^m \|\mathbf{m}_{ij} - \tilde{\mathbf{m}}(\mathbf{A}, k_1, k_2, \mathbf{R}_i, \mathbf{t}_i, \mathbf{M}_j)\|^2$$

Solved with Levenberg-Marquardt algorithm.

Without skew 2 at least images are needed, the more the better.

2/10/2026

Dierksen, Gell - Institute of Computer Science III - Computer Vision Group

83

Solution

UNIVERSITÄT BONN

- Show the plane under n different orientations ($n > 1$)
- Estimate the n homography matrices
(*analytic solution followed by MLE*)
- Solve analytically the 6 intermediate parameters
(*defined up to a scale factor*)
- Extract the five intrinsic parameters
- Compute the extrinsic parameters
- Refine all parameters with MLE

2/10/2026

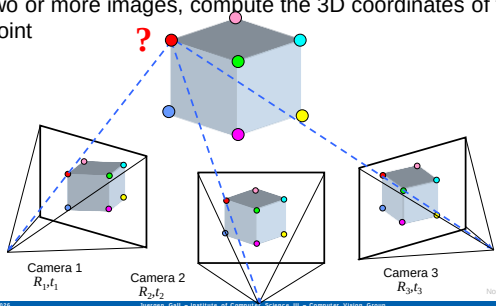
Dierksen, Gell - Institute of Computer Science III - Computer Vision Group

84

Multi-view geometry problems

UNIVERSITÄT BONN

Structure: Given projections of the same 3D point in two or more images, compute the 3D coordinates of that point



2/10/2026

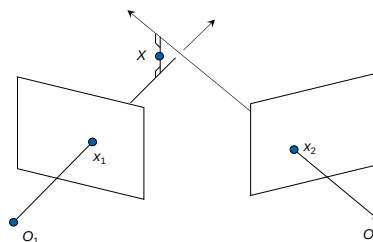
Dierksen, Gell - Institute of Computer Science III - Computer Vision Group

85

Triangulation: Geometric approach

UNIVERSITÄT BONN

Find shortest segment connecting the two viewing rays and let X be the midpoint of that segment



2/10/2026

Dierksen, Gell - Institute of Computer Science III - Computer Vision Group

86

Triangulation: Linear approach



$$\begin{aligned} \lambda_1 x_1 &= P_1 X & x_1 \times P_1 X &= 0 & [x_1]_{\times} P_1 X &= 0 \\ \lambda_2 x_2 &= P_2 X & x_2 \times P_2 X &= 0 & [x_2]_{\times} P_2 X &= 0 \end{aligned}$$

Cross product as matrix multiplication:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix} = [\mathbf{a}]_{\times} \mathbf{b}$$

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

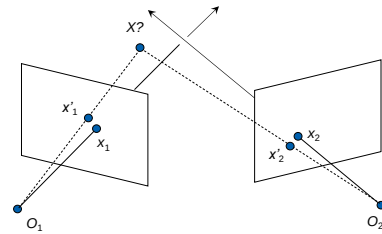
Lecture 6

Triangulation: Nonlinear approach



- Find X that minimizes

$$d^2(x_1, P_1 X) + d^2(x_2, P_2 X)$$

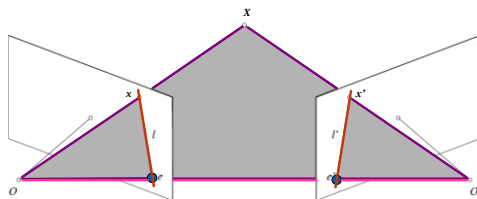


2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 6

Epipolar geometry



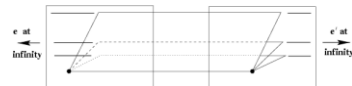
- **Baseline** – line connecting the two camera centers
- **Epipolar Plane** – plane containing baseline (1D family)
- **Epipoles**
= intersections of baseline with image planes
= projections of the other camera center
- **Epipolar Lines** - intersections of epipolar plane with image planes (always come in corresponding pairs)

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 6

Example: Motion parallel to image plane

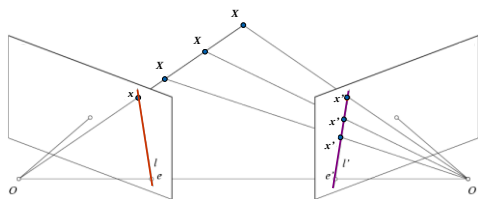


2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 6

Epipolar constraint



Potential matches for x have to lie on the corresponding epipolar line l' .

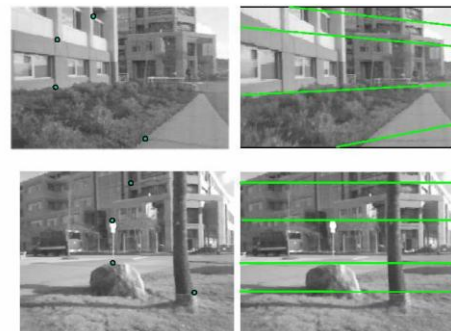
Potential matches for x' have to lie on the corresponding epipolar line l .

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 6

Epipolar constraint example



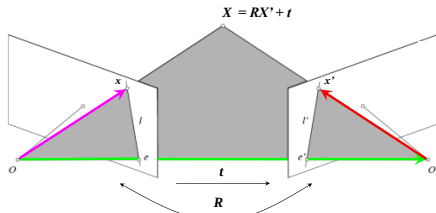
2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 6

From geometry to algebra

UNIVERSITÄT BONN



$$\begin{aligned} \mathbf{X} &= \mathbf{R}\mathbf{X}' + \mathbf{t} \\ \mathbf{T} \times \mathbf{X} &= \mathbf{T} \times \mathbf{R}\mathbf{X}' + \mathbf{T} \times \mathbf{T} \\ &= \mathbf{T} \times \mathbf{R}\mathbf{X}' \end{aligned} \quad \begin{aligned} \mathbf{X} \cdot (\mathbf{T} \times \mathbf{X}) &= \mathbf{X} \cdot (\mathbf{T} \times \mathbf{R}\mathbf{X}') \\ &= 0 \end{aligned}$$

Normal to the plane

2/10/2026

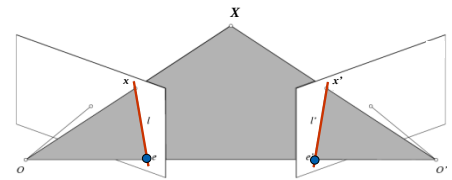
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Klausur Gruppen

72

Epipolar constraint: Calibrated case

UNIVERSITÄT BONN



$$\mathbf{x} \cdot [\mathbf{t} \times (\mathbf{R}\mathbf{x}')] = 0 \Rightarrow \mathbf{x}^T \mathbf{E} \mathbf{x}' = 0 \quad \text{with} \quad \mathbf{E} = [\mathbf{t}_x] \mathbf{R}$$

- $\mathbf{E} \mathbf{x}'$ is the epipolar line associated with $\mathbf{x}'$ ($\mathbf{l} = \mathbf{E} \mathbf{x}'$)
- $\mathbf{E}^T \mathbf{x}$ is the epipolar line associated with $\mathbf{x}$ ($\mathbf{l}' = \mathbf{E}^T \mathbf{x}$)
- $\mathbf{E} \mathbf{e}' = 0$ and $\mathbf{E}^T \mathbf{e} = 0$
- $\mathbf{E}$ is singular (rank two)
- $\mathbf{E}$ has five degrees of freedom

2/10/2026

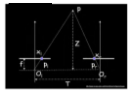
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Klausur

73

Essential matrix example: parallel cameras

UNIVERSITÄT BONN



$$\begin{aligned} \mathbf{R} &= \mathbf{I} \\ \mathbf{T} &= [-d, 0, 0]^T \\ \mathbf{E} &= [\mathbf{T}_x] \mathbf{R} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & d \\ 0 & -d & 0 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} \mathbf{p} &= [x, y, f]^T \\ \mathbf{p}' &= [x', y', f]^T \end{aligned}$$

$$\mathbf{p}^T \mathbf{E} \mathbf{p} = 0 \quad \begin{bmatrix} x' & y' & f \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & d \\ 0 & -d & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ f \end{bmatrix} = 0$$

For the parallel cameras, image of any point must lie on same horizontal line in each image plane.

$$\Leftrightarrow \begin{bmatrix} x' & y' & f \end{bmatrix} \begin{bmatrix} 0 \\ df \\ -dy \end{bmatrix} = 0 \quad \Leftrightarrow y = y'$$

2/10/2026

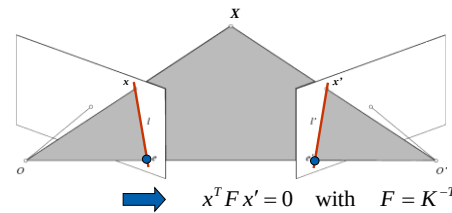
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Klausur Gruppen

74

Epipolar constraint: Uncalibrated case

UNIVERSITÄT BONN



$$\mathbf{x}^T \mathbf{F} \mathbf{x}' = 0 \quad \text{with} \quad \mathbf{F} = \mathbf{K}^{-T} \mathbf{E} \mathbf{K}'^{-1}$$

- $\mathbf{F} \mathbf{x}'$ is the epipolar line associated with $\mathbf{x}'$ ($\mathbf{l} = \mathbf{F} \mathbf{x}'$)
- $\mathbf{F}^T \mathbf{x}$ is the epipolar line associated with $\mathbf{x}$ ($\mathbf{l}' = \mathbf{F}^T \mathbf{x}$)
- $\mathbf{F} \mathbf{e}' = 0$ and $\mathbf{F}^T \mathbf{e} = 0$
- $\mathbf{F}$ is singular (rank two)
- $\mathbf{F}$ has seven degrees of freedom

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Klausur

75

The eight-point algorithm

UNIVERSITÄT BONN

$$\mathbf{x} = (u, v, 1)^T, \quad \mathbf{x}' = (u', v', 1)^T$$

$$(u, v, 1) \begin{pmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{pmatrix} \begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix} = 0 \Rightarrow (uu', uv', u, vv', v, v', u', v', 1) \begin{pmatrix} F_{11} \\ F_{12} \\ F_{13} \\ F_{21} \\ F_{22} \\ F_{23} \\ F_{31} \\ F_{32} \\ F_{33} \end{pmatrix} = 0$$

$$\begin{pmatrix} u_1 u'_1 & u_1 v'_1 & u_1 & v_1 u'_1 & v_1 v'_1 & v_1 & u'_1 & v'_1 & 1 \\ u_2 u'_2 & u_2 v'_2 & u_2 & v_2 u'_2 & v_2 v'_2 & v_2 & u'_2 & v'_2 & 1 \\ u_3 u'_3 & u_3 v'_3 & u_3 & v_3 u'_3 & v_3 v'_3 & v_3 & u'_3 & v'_3 & 1 \\ u_4 u'_4 & u_4 v'_4 & u_4 & v_4 u'_4 & v_4 v'_4 & v_4 & u'_4 & v'_4 & 1 \\ u_5 u'_5 & u_5 v'_5 & u_5 & v_5 u'_5 & v_5 v'_5 & v_5 & u'_5 & v'_5 & 1 \\ u_6 u'_6 & u_6 v'_6 & u_6 & v_6 u'_6 & v_6 v'_6 & v_6 & u'_6 & v'_6 & 1 \\ u_7 u'_7 & u_7 v'_7 & u_7 & v_7 u'_7 & v_7 v'_7 & v_7 & u'_7 & v'_7 & 1 \\ u_8 u'_8 & u_8 v'_8 & u_8 & v_8 u'_8 & v_8 v'_8 & v_8 & u'_8 & v'_8 & 1 \end{pmatrix} \begin{pmatrix} F_{11} \\ F_{12} \\ F_{13} \\ F_{21} \\ F_{22} \\ F_{23} \\ F_{31} \\ F_{32} \\ F_{33} \end{pmatrix} = - \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

Minimize:

$$\sum_{i=1}^N (\mathbf{x}_i^T \mathbf{F} \mathbf{x}'_i)^2$$

under the constraint $F_{33} = 1$

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Klausur

76

The normalized eight-point algorithm

UNIVERSITÄT BONN

(Hartley, 1995)

- Center the image data at the origin, and scale it so the mean squared distance between the origin and the data points is 2 pixels
- Use the eight-point algorithm to compute $\mathbf{F}$ from the normalized points
- Enforce the rank-2 constraint (for example, take SVD of $\mathbf{F}$ and throw out the smallest singular value)
- Transform fundamental matrix back to original units: if $\mathbf{T}$ and $\mathbf{T}'$ are the normalizing transformations in the two images, then the fundamental matrix in original coordinates is $\mathbf{T}^T \mathbf{F} \mathbf{T}'$

2/10/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Klausur

77

From epipolar geometry to camera calibration

- Estimating the fundamental matrix is known as “weak calibration”
- If we know the calibration matrices of the two cameras, we can estimate the essential matrix: $E = K^T F K'$
- The essential matrix gives us the relative rotation and translation between the cameras, or their extrinsic parameters



2/10/2026 Jürgen Gall – Institute of Computer Science III – Computer Vision Group 79

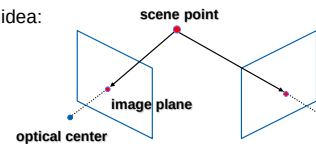
Estimating scene shape

“Shape from X”: Shading, Texture, Focus, Motion...

Stereo:

- shape from “motion” between two views
- infer 3d shape of scene from two (multiple) images from different viewpoints

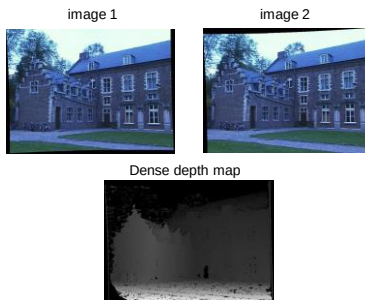
Main idea:



2/10/2026 Jürgen Gall – Institute of Computer Science III – Computer Vision Group 80

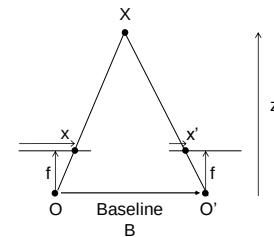
Binocular stereo

Given a calibrated binocular stereo pair, fuse it to produce a depth image



2/10/2026 Jürgen Gall – Institute of Computer Science III – Computer Vision Group 81

Depth from disparity



$$disparity = x - x' = \frac{B \cdot f}{z}$$

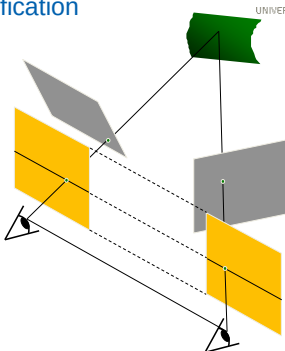
Disparity is inversely proportional to depth!



2/10/2026 Jürgen Gall – Institute of Computer Science III – Computer Vision Group 82

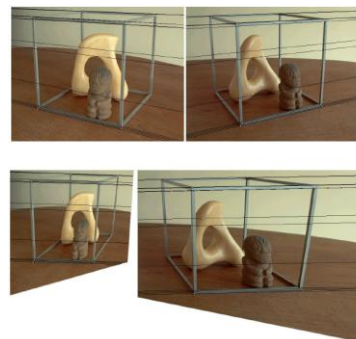
Stereo image rectification

- Reproject image planes onto a common plane parallel to the line between optical centers
- Pixel motion is horizontal after this transformation
- Two homographies (3x3 transform), one for each input image reprojection
- C. Loop and Z. Zhang. Computing Rectifying Homographies for Stereo Vision. IEEE Conf. Computer Vision and Pattern Recognition, 1999.



2/10/2026 Jürgen Gall – Institute of Computer Science III – Computer Vision Group 83

Rectification example



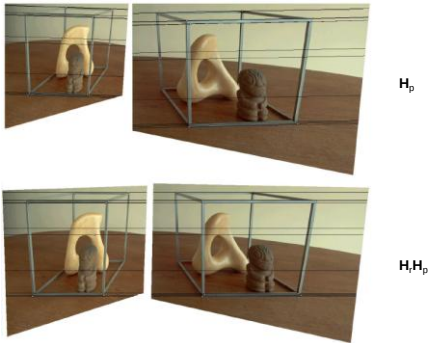
H_p



2/10/2026 Jürgen Gall – Institute of Computer Science III – Computer Vision Group 84

Rectification example

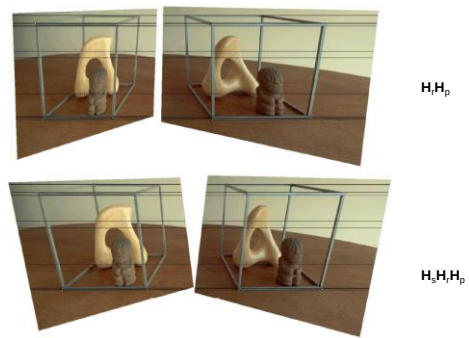
UNIVERSITÄT BONN



2/10/2016 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 14

Rectification example

UNIVERSITÄT BONN



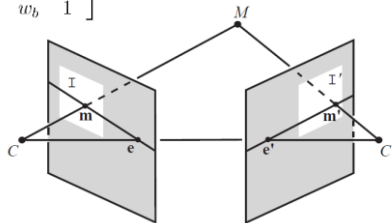
2/10/2016 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 15

Rectification

UNIVERSITÄT BONN

Estimate two homographies H and H'

$$H = \begin{bmatrix} u_a & u_b & u_c \\ v_a & v_b & v_c \\ w_a & w_b & 1 \end{bmatrix}$$



2/10/2016 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 16

Rectification

UNIVERSITÄT BONN

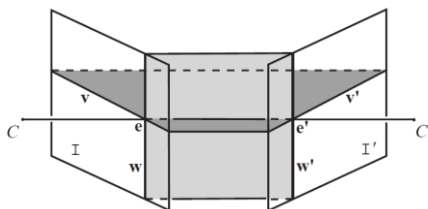
- Map epipoles to infinity (1,0,0) (canonical form)
- Fundamental matrix after rectification:

$$\bar{F} = [i]_{\times} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

2/10/2016 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 17

Rectification

UNIVERSITÄT BONN



$$H = \begin{bmatrix} \mathbf{u}^T \\ \mathbf{v}^T \\ \mathbf{w}^T \end{bmatrix} = \begin{bmatrix} u_a & u_b & u_c \\ v_a & v_b & v_c \\ w_a & w_b & 1 \end{bmatrix}$$

$$He = [\mathbf{u}^T e \quad \mathbf{v}^T e \quad \mathbf{w}^T e]^T = [1 \quad 0 \quad 0]^T$$

2/10/2016 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 18

Decompose homography

UNIVERSITÄT BONN

- Decompose H into affine transformation H_a and projective transformation H_p
- Decompose affine trans. H_a into similarity trans. H_s and shearing trans. H_r
- $H = H_a H_p = H_s H_r H_p$

$$H_s = \begin{bmatrix} s_a & s_b & s_c \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$H_r = \begin{bmatrix} v_b - v_c w_b & v_c w_a - v_a & 0 \\ v_a - v_c w_a & v_b - v_c w_b & v_c \\ 0 & 0 & 1 \end{bmatrix} \quad H_p = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ w_a & w_b & 1 \end{bmatrix}$$

2/10/2016 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 19

Estimate projective transformation



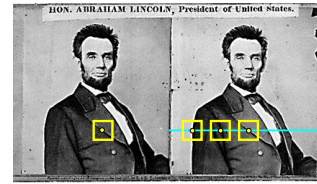
- Estimate H_p :
 - All pixels from images
- $$H_p = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ w_a & w_b & 1 \end{bmatrix}$$
- $$P = \begin{bmatrix} p_{1,u} - p_{c,u} & p_{2,u} - p_{c,u} & \dots & p_{n,u} - p_{c,u} \\ p_{1,v} - p_{c,v} & p_{2,v} - p_{c,v} & \dots & p_{n,v} - p_{c,v} \\ 0 & 0 & \dots & 0 \end{bmatrix} \quad P_c = \frac{1}{n} \sum_{i=1}^n p_i$$
- Minimize: $\frac{z^T \left[\frac{A}{B} \right] P P^T \left[\frac{A}{B} \right] z}{z^T \left[\frac{A'}{B'} \right] P' P'^T \left[\frac{A'}{B'} \right] z} \quad w = [e]_{\times} z \quad w' = Fz$
 - Cholesky decomposition: $A = D^T D$
 - y is eigenvector with highest eigenvalue of $D^{-T} B D^{-1}$
 - w and w' is given by $w = [e]_{\times} z \quad w' = Fz \quad z = D^{-1} y$

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

31

Basic stereo matching algorithm



- If necessary, rectify the two stereo images to transform epipolar lines into scanlines
- For each pixel x in the first image
 - Find corresponding epipolar scanline in the right image
 - Examine all pixels on the scanline and pick the best match x'
 - Compute disparity $x-x'$ and set $depth(x) = B^*f/(x-x')$

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

32

Failures of correspondence search



Textureless surfaces



Occlusions, repetition



Non-Lambertian surfaces, specularities

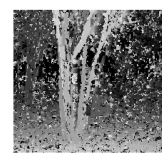


2/10/2024

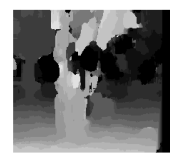
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

33

Effect of window size



W = 3



W = 20

- Smaller window
 - More detail
 - More noise
- Larger window
 - Smoother disparity maps
 - Less detail

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

34

Non-local constraints



Uniqueness

- For any point in one image, there should be at most one matching point in the other image

Ordering

- Corresponding points should be in the same order in both views

Smoothness

- We expect disparity values to change slowly (for the most part)

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

35

Scanline stereo



Try to coherently match pixels on the entire scanline
Different scanlines are still optimized independently



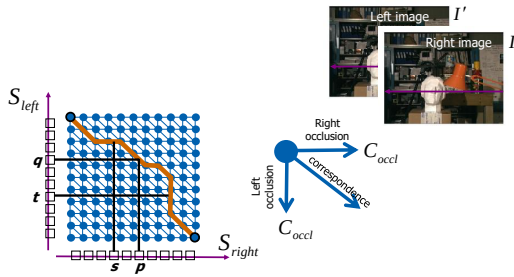
2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

36

"Shortest paths" for scan-line stereo

UNIVERSITÄT BONN



Can be implemented with dynamic programming
Ohta & Kanade '85, Cox et al. '96

2/18/2026

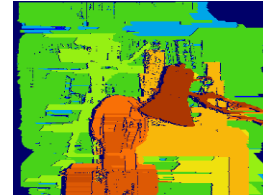
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

87

Coherent stereo on 2D grid

UNIVERSITÄT BONN

Scanline stereo generates streaking artifacts



Can't use dynamic programming to find spatially coherent disparities/ correspondences on a 2D grid

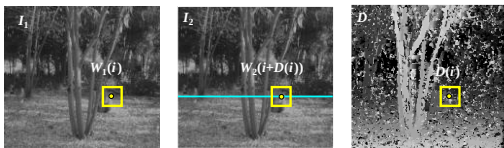
2/18/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

88

Stereo matching as energy minimization

UNIVERSITÄT BONN



$$E(D) = \underbrace{\sum_i (W_1(i) - W_2(i + D(i)))^2}_{\text{data term}} + \lambda \underbrace{\sum_{\text{neighbors } i,j} \rho(D(i) - D(j))}_{\text{smoothness term}}$$

Energy functions of this form can be minimized using graph cuts

Y. Boykov, O. Veksler, and R. Zabih, Fast Approximate Energy Minimization via Graph Cuts, PAMI 2001

2/18/2026

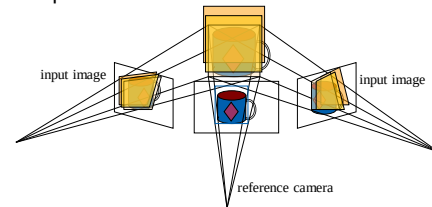
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

89

Plane Sweep Stereo

UNIVERSITÄT BONN

- Choose a reference view
- Sweep family of planes at different depths with respect to the reference camera



Each plane defines a homography warping each input image into the reference view

R. Collins. A space-sweep approach to true multi-image matching. CVPR 1996.

2/18/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

90

Plane Sweep Stereo

UNIVERSITÄT BONN

- For each depth plane
 - For each pixel in the composite image stack, compute the variance



- For each pixel, select the depth that gives the lowest variance

2/18/2026

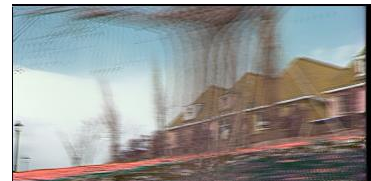
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

91

Plane Sweep Stereo

UNIVERSITÄT BONN

- For each depth plane
 - For each pixel in the composite image stack, compute the variance



- For each pixel, select the depth that gives the lowest variance

- Can be accelerated using graphics hardware

R. Yang and M. Pollefeys. Multi-Resolution Real-Time Stereo on Commodity Graphics Hardware, CVPR 2003

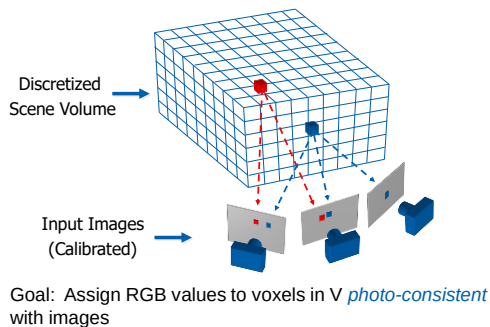
2/18/2026

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

92

Volumetric Stereo / Voxel Coloring

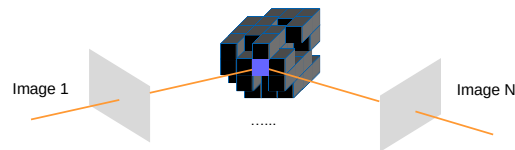
UNIVERSITÄT BONN



2/10/2006 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 182

Space Carving

UNIVERSITÄT BONN



•Space Carving Algorithm

- Initialize to a volume V containing the true scene
- Choose a voxel on the outside of the volume
- Project to visible input images
- Carve if not photo-consistent
- Repeat until convergence

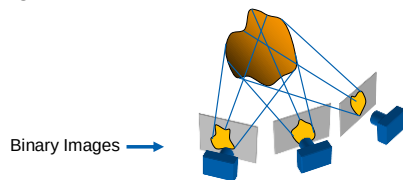
K. N. Kutulakos and S. M. Seitz, **A Theory of Shape by Space Carving**, ICCV 1999

2/10/2006 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 183

Reconstruction from Silhouettes

UNIVERSITÄT BONN

- The case of binary images: a voxel is photo-consistent if it lies inside the object's silhouette in all views



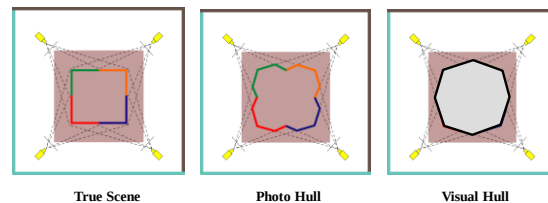
Finding the silhouette-consistent shape (*visual hull*):

- Backproject each silhouette
- Intersect backprojected volumes

2/10/2010 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 184

Photo-consistency vs. silhouette-consistency

UNIVERSITÄT BONN



True Scene

Photo Hull

Visual Hull

2/10/2009 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 185

Carved visual hulls

UNIVERSITÄT BONN

- Compute visual hull
- Use dynamic programming to find rims and constrain them to be fixed
- Carve the visual hull to optimize photo-consistency



Yasutaka Furukawa and Jean Ponce, **Carved Visual Hulls for Image-Based Modeling**, ECCV 2006.

2/10/2010 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 187

Carved visual hulls: Pros and cons

UNIVERSITÄT BONN

- Pros
 - Visual hull gives a reasonable initial mesh that can be iteratively deformed
- Cons
 - Need silhouette extraction
 - Have to compute a lot of points that don't lie on the object
 - Finding rims is difficult
 - The carving step can get caught in local minima
- Possible solution: use sparse feature correspondences as initialization

2/10/2010 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 188

From feature matching to dense stereo

UNIVERSITÄT BONN

1. Extract features
2. Get a sparse set of initial matches
3. Iteratively expand matches to nearby locations
4. Use visibility constraints to filter out false matches
5. Perform surface reconstruction



Yasutaka Furukawa and Jean Ponce, **Accurate, Dense, and Robust Multi-View Stereopsis**, CVPR 2007.

Structure from motion ambiguity

UNIVERSITÄT BONN

If we scale the entire scene by some factor k and, at the same time, scale the camera matrices by the factor of $1/k$, the projections of the scene points in the image remain exactly the same

More generally: if we transform the scene using a transformation Q and apply the inverse transformation to the camera matrices, then the images do not change

$$\mathbf{x} = \mathbf{P}\mathbf{X} = (\mathbf{P}\mathbf{Q}^{-1})(\mathbf{Q}\mathbf{X})$$

2/3/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 9

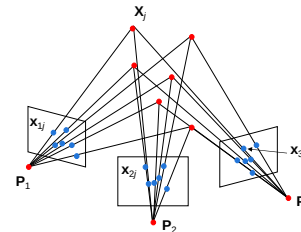
Structure from motion

UNIVERSITÄT BONN

Given: m images of n fixed 3D points

$$\mathbf{x}_{ij} = \mathbf{P}_i \mathbf{X}_j, \quad i = 1, \dots, m, \quad j = 1, \dots, n$$

Problem: estimate m projection matrices $\mathbf{P}_i$ and n 3D points $\mathbf{X}_j$ from the mn correspondences $\mathbf{x}_{ij}$



2/3/2004

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 9

Types of ambiguity

UNIVERSITÄT BONN

Projective 15dof	$\begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{v}^T & v \end{bmatrix}$		Preserves intersection and tangency
Affine 12dof	$\begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$		Preserves parallelism, volume ratios
Similarity 7dof	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$		Preserves angles, ratios of length
Euclidean 6dof	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$		Preserves angles, lengths

- With no constraints on the camera calibration matrix or on the scene, we get a *projective reconstruction*
- Need additional information to *upgrade* the reconstruction to affine, similarity, or Euclidean

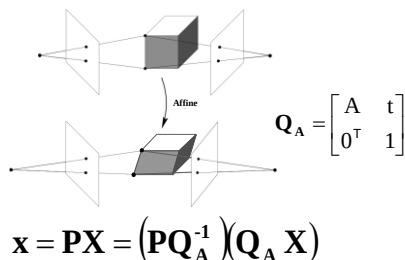
2/3/2005

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 9

Affine ambiguity

UNIVERSITÄT BONN



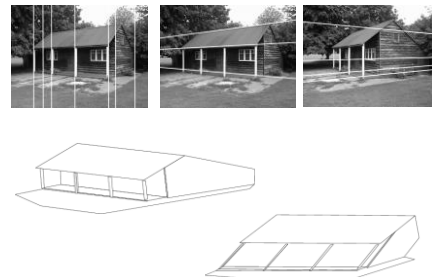
2/3/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

Lecture 9

Affine ambiguity

UNIVERSITÄT BONN



2/3/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

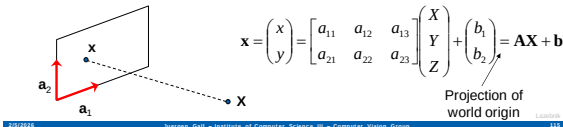
Lecture 9

Affine cameras

- A general affine camera combines the effects of an affine transformation of the 3D space, orthographic projection, and an affine transformation of the image:

$$\mathbf{P} = [\mathbf{3} \times \mathbf{3} \text{ affine}] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} [\mathbf{4} \times \mathbf{4} \text{ affine}] = \begin{bmatrix} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0} & \mathbf{1} \end{bmatrix}$$

- Affine projection is a linear mapping + translation in inhomogeneous coordinates



Affine structure from motion

- Given: m images of n fixed 3D points:

$$\mathbf{x}_{ij} = \mathbf{A}_i \mathbf{X}_j + \mathbf{b}_i, \quad i = 1, \dots, m, \quad j = 1, \dots, n$$
- Problem: use the mn correspondences $\mathbf{x}_{ij}$ to estimate m projection matrices $\mathbf{A}_i$ and translation vectors $\mathbf{b}_i$, and n points $\mathbf{X}_j$
- The reconstruction is defined up to an arbitrary affine transformation $\mathbf{Q}$ (12 degrees of freedom):

$$\begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0} & \mathbf{1} \end{bmatrix} \rightarrow \begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0} & \mathbf{1} \end{bmatrix} \mathbf{Q}^{-1}, \quad \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix} \rightarrow \mathbf{Q} \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix}$$

- We have $2mn$ knowns and $8m + 3n$ unknowns (minus 12 dof for affine ambiguity)
- Thus, we must have $2mn \geq 8m + 3n - 12$
- For two views, we need four point correspondences

Affine structure from motion

- Centering: subtract the centroid of the image points

$$\hat{\mathbf{x}}_{ij} = \mathbf{x}_{ij} - \frac{1}{n} \sum_{k=1}^n \mathbf{x}_{ik} = \mathbf{A}_i \mathbf{X}_j + \mathbf{b}_i - \frac{1}{n} \sum_{k=1}^n (\mathbf{A}_i \mathbf{X}_k + \mathbf{b}_i) \\ = \mathbf{A}_i \left(\mathbf{X}_j - \frac{1}{n} \sum_{k=1}^n \mathbf{X}_k \right) = \mathbf{A}_i \hat{\mathbf{X}}_j$$

- For simplicity, assume that the origin of the world coordinate system is at the centroid of the 3D points
- After centering, each normalized point $\mathbf{x}_{ij}$ is related to the 3D point $\mathbf{X}_j$ by

$$\hat{\mathbf{x}}_{ij} = \mathbf{A}_i \mathbf{X}_j$$

Affine structure from motion

- Let's create a $2m \times n$ data (measurement) matrix:

$$\mathbf{D} = \begin{bmatrix} \hat{\mathbf{x}}_{11} & \hat{\mathbf{x}}_{12} & \dots & \hat{\mathbf{x}}_{1n} \\ \hat{\mathbf{x}}_{21} & \hat{\mathbf{x}}_{22} & \dots & \hat{\mathbf{x}}_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{\mathbf{x}}_{m1} & \hat{\mathbf{x}}_{m2} & \dots & \hat{\mathbf{x}}_{mn} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \\ \vdots \\ \mathbf{A}_m \end{bmatrix} \begin{bmatrix} \mathbf{X}_1 & \mathbf{X}_2 & \dots & \mathbf{X}_n \end{bmatrix}$$

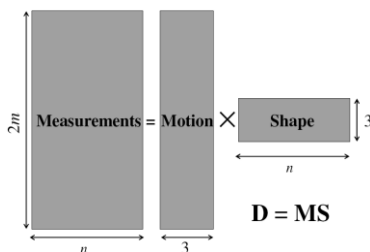
cameras
($2m \times 3$)

points ($3 \times n$)

The measurement matrix $\mathbf{D} = \mathbf{MS}$ must have rank 3!

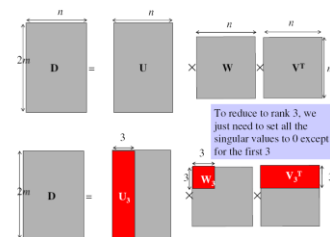
C. Tomasi and T. Kanade, Shape and motion from image streams under orthography: A factorization method. *IJCV*, 9(2):137-154, November 1992.

Factorizing the measurement matrix



Factorizing the measurement matrix

- Singular value decomposition of $\mathbf{D}$:



Factorizing the measurement matrix

- Obtaining a factorization from SVD:

Possible decomposition:
 $\mathbf{M} = \mathbf{U}_3 \mathbf{W}_3^{1/2}$ $\mathbf{S} = \mathbf{W}_3^{1/2} \mathbf{V}_3^T$

This decomposition minimizes $\|\mathbf{D} - \mathbf{MS}\|^2$

2/10/2004 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 122

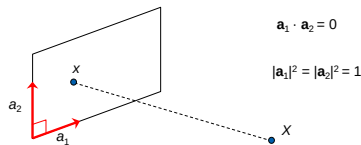
Affine ambiguity

- The decomposition is not unique. We get the same $\mathbf{D}$ by using any 3×3 matrix $\mathbf{C}$ and applying the transformations $\mathbf{M} \rightarrow \mathbf{MC}$, $\mathbf{S} \rightarrow \mathbf{C}^{-1}\mathbf{S}$
- That is because we have only an affine transformation and we have not enforced any Euclidean constraints (like forcing the image axes to be perpendicular, for example)

2/10/2004 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 123

Eliminating the affine ambiguity

- Orthographic: image axes are perpendicular and of unit length



2/10/2004 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 124

Solve for orthographic constraints

Three equations for each image i

$$\begin{aligned} \mathbf{m}_{i1}^T \mathbf{C} \mathbf{C}^T \mathbf{m}_{i1} &= 1 \\ \mathbf{m}_{i2}^T \mathbf{C} \mathbf{C}^T \mathbf{m}_{i2} &= 1 \\ \mathbf{m}_{i1}^T \mathbf{C} \mathbf{C}^T \mathbf{m}_{i2} &= 0 \end{aligned} \quad \text{where } \mathbf{M}_i = \begin{bmatrix} \mathbf{m}_{i1}^T \\ \mathbf{m}_{i2}^T \end{bmatrix}$$

- Two options:
 - Solve for $\mathbf{C}$ (Newton's method, quadratic)
 - Solve linearly $\mathbf{L} = \mathbf{C} \mathbf{C}^T$
 - Recover $\mathbf{C}$ from $\mathbf{L}$ by SVD or Cholesky decomposition: $\mathbf{L} = \mathbf{C} \mathbf{C}^T$
- Update $\mathbf{M}$ and $\mathbf{S}$: $\mathbf{M}' = \mathbf{MC}$, $\mathbf{S}' = \mathbf{C}^{-1}\mathbf{S}$

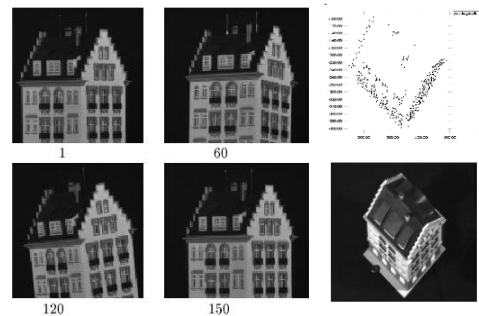
2/10/2004 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 125

Algorithm summary

- Given: m images and n features $\mathbf{x}_{ij}$
- For each image i , center the feature coordinates
- Construct a $2m \times n$ measurement matrix $\mathbf{D}$:
 - Column j contains the projection of point j in all views
 - Row i contains one coordinate of the projections of all the n points in image i
- Factorize $\mathbf{D}$:
 - Compute SVD: $\mathbf{D} = \mathbf{U} \mathbf{W} \mathbf{V}^T$
 - Create $\mathbf{U}_3$ by taking the first 3 columns of $\mathbf{U}$
 - Create $\mathbf{V}_3$ by taking the first 3 columns of $\mathbf{V}$
 - Create $\mathbf{W}_3$ by taking the upper left 3×3 block of $\mathbf{W}$
- Create the motion and shape matrices:
 - $\mathbf{M} = \mathbf{U}_3 \mathbf{W}_3^{1/2}$ and $\mathbf{S} = \mathbf{W}_3^{1/2} \mathbf{V}_3^T$ (or $\mathbf{M} = \mathbf{U}_3$ and $\mathbf{S} = \mathbf{W}_3 \mathbf{V}_3^T$)
- Eliminate affine ambiguity

2/10/2004 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 126

Reconstruction results



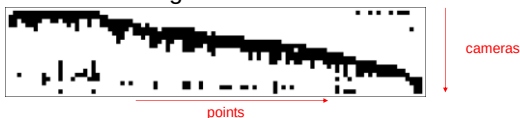
C. Tomasi and T. Kanade. Shape and motion from image streams under orthography: A factorization method. *IJCV*, 9(2):137-154, November 1992.

2/10/2004 Jürgen Gall - Institute of Computer Science III - Computer Vision Group 127

Dealing with missing data

UNIVERSITÄT BONN

- So far, we have assumed that all points are visible in all views
- In reality, the measurement matrix typically looks something like this:



2/10/2024

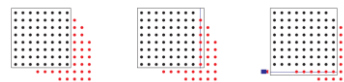
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

127

Dealing with missing data

UNIVERSITÄT BONN

- Possible solution: decompose matrix into dense sub-blocks, factorize each sub-block, and fuse the results
 - Finding dense maximal sub-blocks of the matrix is NP-complete (equivalent to finding maximal cliques in a graph)
- Incremental bilinear refinement



- (1) Perform factorization on a dense sub-block
- (2) Solve for a new 3D point visible by at least two known cameras (linear least squares)
- (3) Solve for a new camera that sees at least three known 3D points (linear least squares)

F. Rothganger, S. Lazebnik, C. Schmid, and J. Ponce. Segmenting, Modeling, and Matching Video Clips Containing Multiple Moving Objects. PAMI 2007.

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

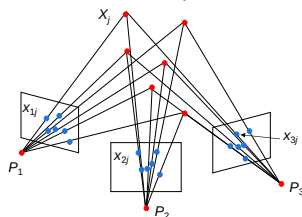
128

Projective structure from motion

UNIVERSITÄT BONN

- Given: m images of n fixed 3D points

$$z_{ij} \mathbf{x}_{ij} = \mathbf{P}_i \mathbf{X}_j, \quad i = 1, \dots, m, \quad j = 1, \dots, n$$
- Problem: estimate m projection matrices $\mathbf{P}_i$ and n 3D points $\mathbf{X}_j$ from the mn correspondences $\mathbf{x}_{ij}$



2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

129

Projective structure from motion

UNIVERSITÄT BONN

- Given: m images of n fixed 3D points

$$z_{ij} \mathbf{x}_{ij} = \mathbf{P}_i \mathbf{X}_j, \quad i = 1, \dots, m, \quad j = 1, \dots, n$$
- Problem: estimate m projection matrices $\mathbf{P}_i$ and n 3D points $\mathbf{X}_j$ from the mn correspondences $\mathbf{x}_{ij}$
- With no calibration info, cameras and points can only be recovered up to a 4x4 projective transformation $\mathbf{Q}$:

$$\mathbf{X} \rightarrow \mathbf{QX}, \mathbf{P} \rightarrow \mathbf{PQ}^{-1}$$
- We can solve for structure and motion when

$$2mn \geq 11m + 3n - 15$$
- For two cameras, at least 7 points are needed

2/10/2024

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

130

Projective SFM: Two-camera case

UNIVERSITÄT BONN

- Compute fundamental matrix $\mathbf{F}$ between the two views
- First camera matrix: $[\mathbf{I}|\mathbf{0}]$
- Second camera matrix: $[\mathbf{A}|\mathbf{b}]$
- Then $\mathbf{b}$ is the epipole ($\mathbf{F}^T \mathbf{b} = \mathbf{0}$), $\mathbf{A} = -[\mathbf{b}_\times] \mathbf{F}$

2/10/2024

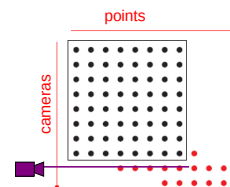
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

131

Sequential structure from motion

UNIVERSITÄT BONN

- Initialize motion from two images using fundamental matrix
- Initialize structure by triangulation
- For each additional view:
 - Determine projection matrix of new camera using all the known 3D points that are visible in its image – *calibration*



2/10/2024

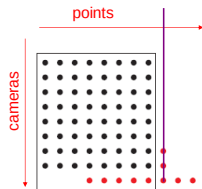
Jürgen Gall - Institute of Computer Science III - Computer Vision Group

132

Sequential structure from motion



- Initialize motion from two images using fundamental matrix
- Initialize structure by triangulation
- For each additional view:
 - Determine projection matrix of new camera using all the known 3D points that are visible in its image – calibration
 - Refine and extend structure: compute new 3D points, re-optimize existing points that are also seen by this camera – triangulation



2/10/2025

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

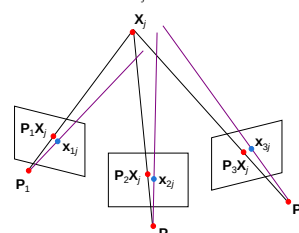
132

Bundle adjustment



- Non-linear method for refining structure and motion
- Minimizing reprojection error

$$E(\mathbf{P}, \mathbf{X}) = \sum_{i=1}^m \sum_{j=1}^n w_{ij} D(\mathbf{x}_{ij}, \mathbf{P}_i \mathbf{X}_{ij})^2$$

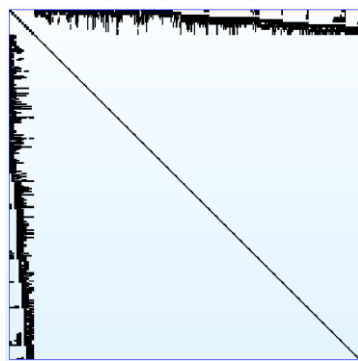


2/10/2025

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

134

Hessian (real problem)



Black: non-zero

2/10/2025

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

135

Self-calibration



- Self-calibration (auto-calibration) is the process of determining intrinsic camera parameters directly from uncalibrated images
- For example, when the images are acquired by a single moving camera, we can use the constraint that the intrinsic parameter matrix remains fixed for all the images
 - Compute initial projective reconstruction and find 3D projective transformation matrix $\mathbf{Q}$ such that all camera matrices are in the form $\mathbf{P}_i = \mathbf{K} [\mathbf{R}_i | \mathbf{t}_i]$
- Can use constraints on the form of the calibration matrix: zero skew

2/10/2025

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

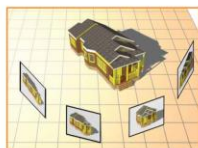
136

FACTORIZATION METHOD FOR RIGID SFM

Kontsevich *et al.* 1987, Tomasi and Kanade 1992

ASSUMPTIONS

- Orthographic Camera
- At least 3 images
- Rigid Scene
- Camera Motion
- Corresponding points available



2/10/2025

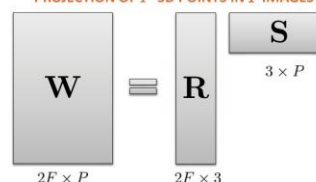
Jürgen Gall – Institute of Computer Science III – Computer Vision Group

137

FACTORIZATION METHOD FOR RIGID SFM

Kontsevich *et al.* 1987, Tomasi and Kanade 1992

PROJECTION OF P 3D POINTS IN F IMAGES



$$\mathbf{W}_{\text{measurement}} = \mathbf{R}_{\text{motion}} \times \mathbf{S}_{\text{shape}}$$

2/10/2025

Jürgen Gall – Institute of Computer Science III – Computer Vision Group

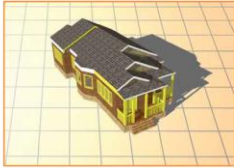
138

NONRIGID STRUCTURE

3D Structure That Deforms Over Time

RIGID STRUCTURE

$$\mathbf{S}_{3 \times P} = \begin{bmatrix} X_1 & X_2 & \dots & X_P \\ Y_1 & Y_2 & \dots & Y_P \\ Z_1 & Z_2 & \dots & Z_P \end{bmatrix}$$



2/10/2006

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

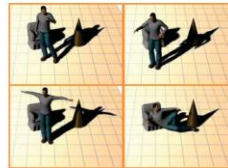
129

NONRIGID STRUCTURE

3D Structure That Deforms Over Time

RIGID STRUCTURE

$$\mathbf{S}_{3 \times P} = \begin{bmatrix} X_1 & X_2 & \dots & X_P \\ Y_1 & Y_2 & \dots & Y_P \\ Z_1 & Z_2 & \dots & Z_P \end{bmatrix}$$



NONRIGID STRUCTURE

$$\mathbf{S}_{3F \times P} = \begin{bmatrix} \begin{bmatrix} X_{11} & X_{12} & \dots & X_{1P} \\ Y_{11} & Y_{12} & \dots & Y_{1P} \\ Z_{11} & Z_{12} & \dots & Z_{1P} \end{bmatrix}_{3 \times P} \\ \begin{bmatrix} X_{21} & X_{22} & \dots & X_{2P} \\ Y_{21} & Y_{22} & \dots & Y_{2P} \\ Z_{21} & Z_{22} & \dots & Z_{2P} \end{bmatrix}_{3 \times P} \\ \vdots \\ \begin{bmatrix} X_{F1} & X_{F2} & \dots & X_{FP} \\ Y_{F1} & Y_{F2} & \dots & Y_{FP} \\ Z_{F1} & Z_{F2} & \dots & Z_{FP} \end{bmatrix}_{3 \times P} \end{bmatrix}$$

2/10/2006

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

130

NONRIGID STRUCTURE FROM MOTION

Comparison with Rigid Structure from Motion

RIGID SFM

$$\mathbf{W} = \mathbf{R} \mathbf{S}$$

$2F \times P \quad 2F \times 3 \quad 3 \times P$

$$\text{Rank}(\mathbf{W}) \leq 3$$

NONRIGID SFM

$$\mathbf{W} = \begin{bmatrix} \mathbf{R}_1 & & & \\ & \mathbf{R}_2 & & \\ & & \ddots & \\ & & & \mathbf{R}_F \end{bmatrix} \mathbf{S}$$

$2F \times P \quad 2F \times 3F \quad 3F \times P$

$$\text{Rank}(\mathbf{W}) \leq \min(2F, P)$$

2/10/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

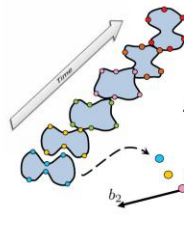
131

NONRIGID STRUCTURE FROM MOTION

Two Major Approaches

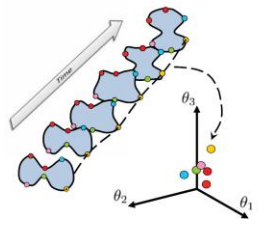
Shape Basis

3D points at each time instant lie in a low dimensional subspace



Trajectory Basis

Trajectory of each point over time lies in a low dimensional subspace



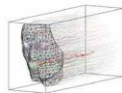
2/10/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

132

LINEAR SHAPE MODEL

[T. Cootes et al. 91, Bregler et al. 97]



$$\begin{bmatrix} \mathbf{X}_{11} & \dots & \mathbf{X}_{1P} \\ \mathbf{X}_{21} & & \mathbf{X}_{2P} \\ \vdots & & \vdots \\ \mathbf{X}_{F1} & \dots & \mathbf{X}_{FP} \end{bmatrix}$$

$$= \alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2 + \dots + \alpha_k \mathbf{x}_k$$

2/10/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

133

LINEAR SHAPE MODEL

$$\begin{bmatrix} \mathbf{X}_{11} & \dots & \mathbf{X}_{1P} \\ \mathbf{X}_{21} & & \mathbf{X}_{2P} \\ \vdots & & \vdots \\ \mathbf{X}_{F1} & \dots & \mathbf{X}_{FP} \end{bmatrix} = \begin{bmatrix} \omega_{11} & \dots & \omega_{1k} \\ \omega_{21} & & \omega_{2k} \\ \vdots & & \vdots \\ \omega_{F1} & \dots & \omega_{Fk} \end{bmatrix} \begin{bmatrix} -\mathbf{b}_1- \\ -\mathbf{b}_2- \\ \vdots \\ -\mathbf{b}_k- \end{bmatrix}$$

2/10/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

134

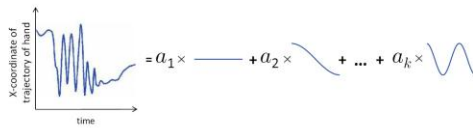
DYNAMIC STRUCTURE

Trajectory Representation



$$S_{3P \times P} = \begin{bmatrix} \mathbf{x}_{11} & \mathbf{x}_{12} & \dots & \mathbf{x}_{1P} \\ \mathbf{x}_{21} & \mathbf{x}_{22} & \dots & \mathbf{x}_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{P1} & \mathbf{x}_{P2} & \dots & \mathbf{x}_{PP} \end{bmatrix} \quad \text{Trajectory}$$

LINEAR TRAJECTORY MODEL



2/10/2006

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

101

DUALITY

Weights and Bases

SHAPE FACTORIZATION

$$\mathbf{W} = \mathbf{R} \begin{bmatrix} \Omega \\ \text{Weights} \end{bmatrix} \mathbf{B} \quad \text{Shape basis}$$

TRAJECTORY FACTORIZATION

$$\mathbf{W} = \mathbf{R} \begin{bmatrix} \Theta \\ \text{Traj basis} \end{bmatrix} \mathbf{A} \quad \text{Weights}$$

Shape weights are trajectory basis and trajectory weights are shape basis

2/10/2006

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

110

2/10/2010

Jürgen Gall - Institute of Computer Science III - Computer Vision Group

111